

MACHINE VISION ASSIGNMENT 4

Sreehitha Korrapati-7/15/2026

KSU ID: 001228797

ABSTRACT:

This assignment implements and compares transfer learning approaches for classifying retinal fundus images as diabetic retinopathy (DR) or non-DR. Following the assignment specification, IDRI images with severity grade 0 were labeled non-DR and grades 3-4 were labeled DR, while grades 1-2 were excluded from this binary task. The goal was to evaluate how well CNNs pretrained on ImageNet adapt to this medical imaging task, and how much architecture choice and hyperparameter selection affect performance. Three pretrained networks, AlexNet, ResNet18, and VGG16, were each retrained across three learning rates (0.0001, 0.001, 0.01) and three epoch counts (5, 10, 20), for 27 configurations per network and 81 total training runs, then evaluated on a held-out test set of 66 images. VGG16 achieved the highest test accuracy at 89.4%, followed by ResNet18 at 84.9% and AlexNet at 80.3%. A consistent pattern emerged across all three architectures: a learning rate of 0.0001 produced the best results, while 0.01 was too aggressive, causing training to diverge for AlexNet and VGG16 and a substantial accuracy drop for ResNet18. These results demonstrate both the practicality of transfer learning for small medical imaging datasets and the importance of careful learning rate selection when fine-tuning pretrained models.

TEST RESULT:

The IDRI training set contained 123 DR and 134 non-DR images after filtering; the test set contained 32 DR and 34 non-DR images. All three networks were retrained using stochastic gradient descent with momentum (0.9), a batch size of 16, and light data augmentation (random horizontal flip, $\pm 10^\circ$ rotation) on the training set. Each network's final classification layer was replaced with a 2-class output layer while all other pretrained ImageNet weights were used as the starting point (transfer learning). The table below summarizes every one of the 27 (learning rate x epoch) configurations tested per network, sorted by test accuracy.

Network	Learning Rate	Epochs	Test Accuracy
vgg16	0.0001	5	0.8939
vgg16	0.0001	20	0.8939
vgg16	0.0001	10	0.8939
vgg16	0.001	10	0.8788
vgg16	0.001	20	0.8788
resnet18	0.001	20	0.8485
resnet18	0.0001	10	0.8485
resnet18	0.001	10	0.8485
resnet18	0.0001	20	0.8333
resnet18	0.0001	5	0.8182
alexnet	0.0001	10	0.8030
alexnet	0.0001	5	0.7879
resnet18	0.001	5	0.7727
alexnet	0.0001	20	0.7576
resnet18	0.01	5	0.7273
alexnet	0.001	20	0.7121
alexnet	0.001	10	0.7121
vgg16	0.001	5	0.6818
alexnet	0.001	5	0.5152
resnet18	0.01	10	0.5152

Network	Learning Rate	Epochs	Test Accuracy
resnet18	0.01	20	0.4848
alexnet	0.01	5	NaN (diverged)
alexnet	0.01	10	NaN (diverged)
alexnet	0.01	20	NaN (diverged)
vgg16	0.01	5	NaN (diverged)
vgg16	0.01	10	NaN (diverged)
vgg16	0.01	20	NaN (diverged)

Table 1. Test accuracy for all 27 (learning rate, epoch) combinations across AlexNet, ResNet18, and VGG16. Rows marked "NaN (diverged)" indicate training loss became NaN (numerical instability) and training was halted early.

AlexNet

AlexNet's best configuration was a learning rate of $1e-4$ with 10 epochs, achieving 80.3% test accuracy. At a learning rate of $1e-2$, training diverged (NaN loss) at all three epoch settings.

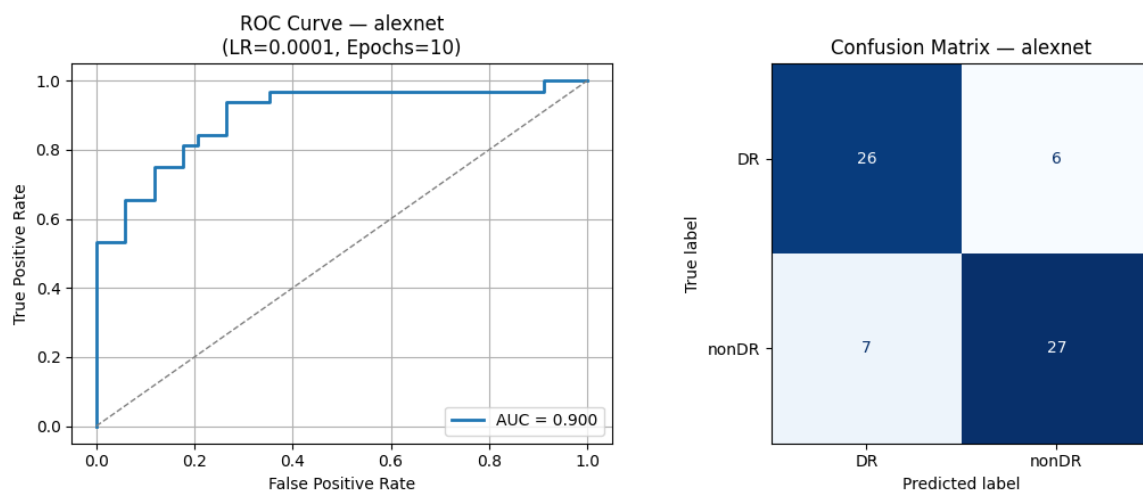


Figure 1. ROC curve (left) and confusion matrix (right) for AlexNet at its best configuration: learning rate = $1e-4$, epochs = 10, test accuracy = 80.3%.

ResNet18

ResNet18's best configuration was a learning rate of $1e-3$ with 20 epochs, achieving 84.9% test accuracy (tied with two other configurations at the same accuracy). Unlike AlexNet and VGG16, ResNet18 did not fully diverge at a learning rate of $1e-2$, though accuracy dropped substantially (to as low as 48.5%), suggesting its residual connections and batch normalization provide some resilience to aggressive learning rates.

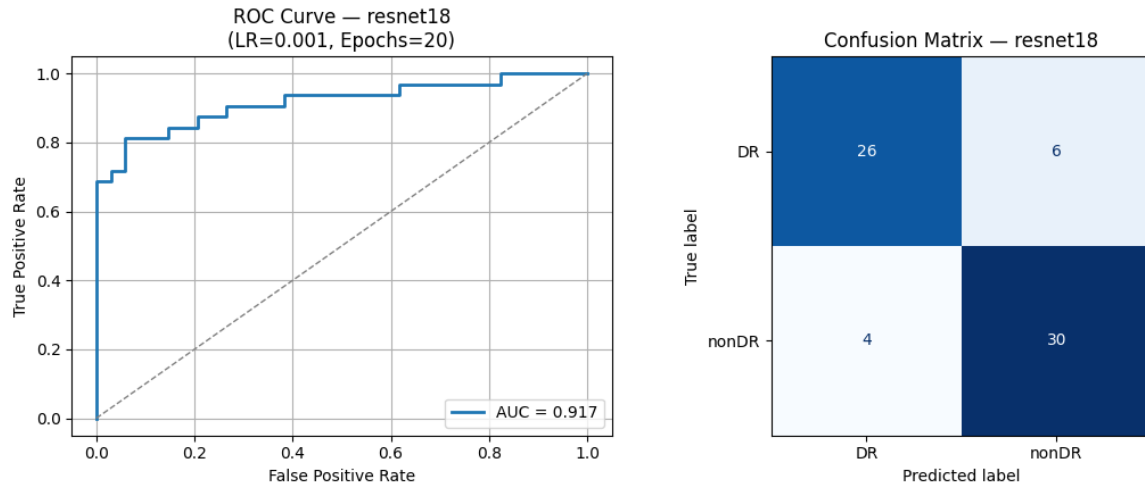


Figure 2. ROC curve (left) and confusion matrix (right) for ResNet18 at its best configuration: learning rate = $1e-3$, epochs = 20, test accuracy = 84.9%.

VGG16

VGG16 achieved the highest overall test accuracy of the three networks: 89.4%, at a learning rate of $1e-4$ (tied across 5, 10, and 20 epochs, indicating fast, stable convergence). Like AlexNet, VGG16 fully diverged at a learning rate of $1e-2$.

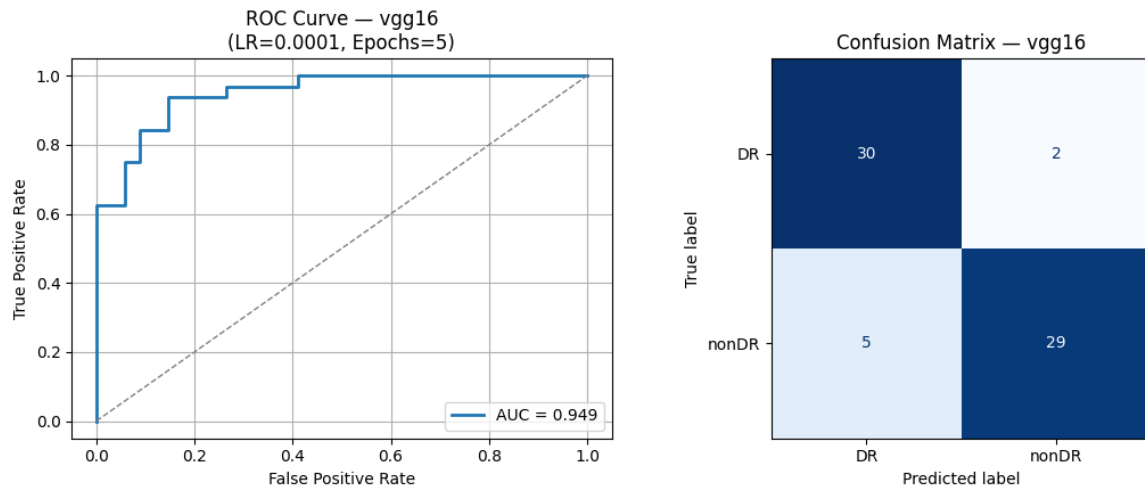


Figure 3. ROC curve (left) and confusion matrix (right) for VGG16 at its best configuration: learning rate = $1e-4$, epochs = 5, test accuracy = 89.4%.

Comparison Across Networks

The table below summarizes each network's best and mean test accuracy across all 9 of its (learning rate, epoch) configurations. VGG16 achieved both the highest best-case accuracy and the highest mean accuracy, indicating it was not only capable of the best single result but also the most consistently high-performing across the hyperparameter grid.

Network	Best Test Accuracy	Mean Test Accuracy
VGG16	0.8939	0.8536

Network	Best Test Accuracy	Mean Test Accuracy
ResNet18	0.8485	0.7441
AlexNet	0.8030	0.7146

Table 2. Best and mean test accuracy per network architecture, computed across all 9 learning-rate/epoch combinations tested for that network.

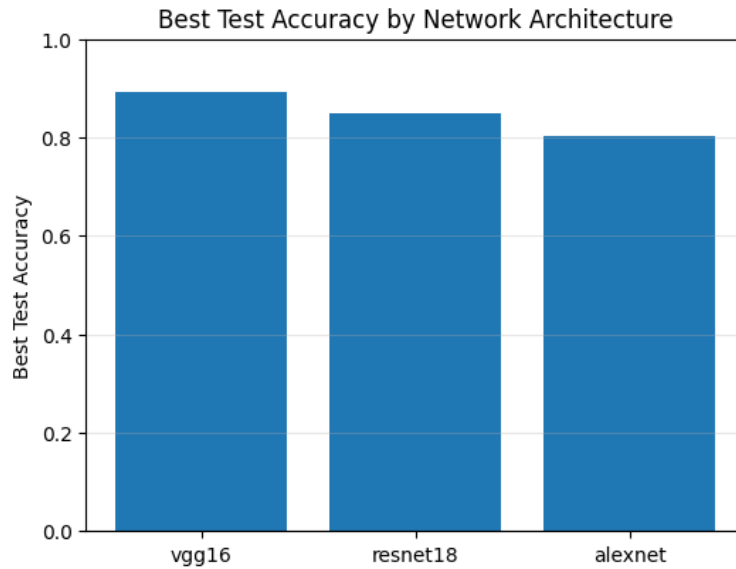


Figure 4. Bar chart comparing the best test accuracy achieved by each of the three network architectures.

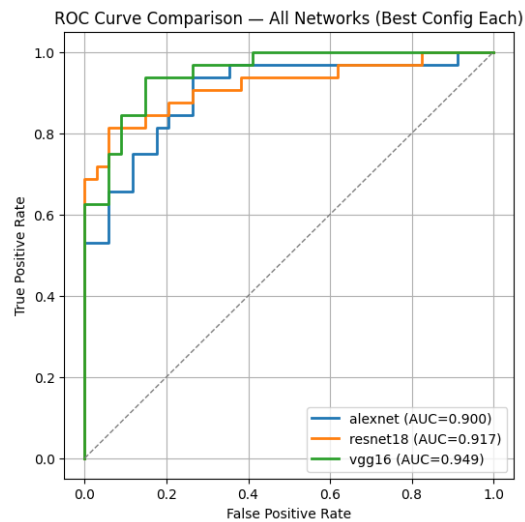


Figure 5. ROC curves for all three networks overlaid, each using its own best-performing (learning rate, epoch) configuration, with AUC values shown in the legend.

DISCUSSION

This assignment reinforced how effective transfer learning can be on a limited dataset. With only 257 training images, all three networks reached 75-89% test accuracy after fine-tuning, which supports why transfer learning is generally

preferred over training from scratch for small, specialized datasets like medical imaging. The most consistent finding was the sensitivity to learning rate: 0.0001 worked best across all architectures, while 0.01 caused training to diverge for AlexNet and VGG16 and reduced accuracy sharply for ResNet18. This makes sense given that fine-tuning starts from weights that already encode useful features, so an aggressive learning rate risks overwriting that progress before the model adapts. It was also notable that VGG16 reached its best accuracy in just 5 epochs despite being the deepest network, suggesting pretrained feature quality mattered more than depth here. A practical lesson came from a training run crashing due to an out-of-memory error mid-sweep, which led me to add error handling so a single failure wouldn't derail the rest of the experiment. Given more time, I would extend this work by using the excluded grade 1-2 images to build a full 5-class severity classifier, applying cross-validation for more reliable accuracy estimates, and testing class-weighted loss to address the mild training set imbalance.

CODE:

Setup

```
import os, re, shutil
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import models, datasets, transforms
from sklearn.metrics import roc_curve, auc, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

os.makedirs('/kaggle/working/results', exist_ok=True)
```

1. Organize raw images into DR / nonDR folders

```
TRAIN_SRC = '/kaggle/input/datasets/sreehithakorrapati/ds-idrid/DS_IDRID/Train'
TEST_SRC = '/kaggle/input/datasets/sreehithakorrapati/ds-idrid/DS_IDRID/Test'
```

```
def organize_data(src_folder, out_folder):
    dr_folder = os.path.join(out_folder, 'DR')
    nondr_folder = os.path.join(out_folder, 'nonDR')
    os.makedirs(dr_folder, exist_ok=True)
    os.makedirs(nondr_folder, exist_ok=True)

    n_dr, n_nondr, n_skip = 0, 0, 0
    for fname in os.listdir(src_folder):
        if not fname.lower().endswith(('.jpg', '.jpeg')):
            continue
        m = re.search(r'-(d+)\.jpe?g$', fname)
        if not m:
```

```

        n_skip += 1
        continue
    label = int(m.group(1))
    src_path = os.path.join(src_folder, fname)
    if label == 0:
        shutil.copy(src_path, os.path.join(nondr_folder, fname))
        n_nondr += 1
    elif label in (3, 4):
        shutil.copy(src_path, os.path.join(dr_folder, fname))
        n_dr += 1
    else:
        n_skip += 1 # labels 1 and 2 are excluded from this binary task

print(f'{src_folder} -> DR: {n_dr}, nonDR: {n_nondr}, skipped: {n_skip}')
return out_folder

train_sorted = organize_data(TRAIN_SRC, '/kaggle/working/sorted_data/Train')
test_sorted = organize_data(TEST_SRC, '/kaggle/working/sorted_data/Test')

```

2. Data loading helpers

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
```

```
IMAGENET_STD = [0.229, 0.224, 0.225]
```

```

def get_train_transform(input_size):
    return transforms.Compose([
        transforms.Resize((input_size, input_size)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(10),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

```

```

def get_test_transform(input_size):
    return transforms.Compose([
        transforms.Resize((input_size, input_size)),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

```

```
# Peek at class-to-index mapping (alphabetical: DR=0, nonDR=1)
```

```

_peek = datasets.ImageFolder(train_sorted)
print('Classes:', _peek.classes, '-> indices', _peek.class_to_idx)

```

3. Model builder

```

def build_model(net_name, num_classes=2):
    if net_name == 'alexnet':
        model = models.alexnet(weights=models.AlexNet_Weights.IMAGENET1K_V1)
        model.classifier[6] = nn.Linear(model.classifier[6].in_features, num_classes)
        input_size = 224
    elif net_name == 'resnet18':
        model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
        model.fc = nn.Linear(model.fc.in_features, num_classes)
        input_size = 224
    elif net_name == 'vgg16':
        model = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
        model.classifier[6] = nn.Linear(model.classifier[6].in_features, num_classes)
        input_size = 224
    else:
        raise ValueError(f'Unrecognized network: {net_name}')
    return model, input_size

```

4. Train + evaluate one (network, LR, epochs) configuration

```

def train_and_evaluate(net_name, lr, epochs, batch_size=16):
    model, input_size = build_model(net_name)
    model = model.to(device)

    train_ds = datasets.ImageFolder(train_sorted, transform=get_train_transform(input_size))
    test_ds = datasets.ImageFolder(test_sorted, transform=get_test_transform(input_size))
    classes = train_ds.classes # e.g. ['DR', 'nonDR']

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)

    model.train()
    for epoch in range(epochs):
        for imgs, labels in train_loader:
            imgs, labels = imgs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            if torch.isnan(loss):
                print(f' NaN loss at epoch {epoch + 1} — training diverged, stopping this run.')
                return None
            loss.backward()
            optimizer.step()

    model.eval()

```

```

all_preds, all_labels, all_scores = [], [], []
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        probs = torch.softmax(outputs, dim=1)
        preds = torch.argmax(probs, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())
        all_scores.extend(probs.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)
all_scores = np.array(all_scores)
acc = float(np.mean(all_preds == all_labels))

return {
    'model': model, 'acc': acc,
    'preds': all_preds, 'labels': all_labels, 'scores': all_scores,
    'classes': classes,
}

```

5. Hyperparameter sweep: networks x learning rates x epochs

```

networks_to_compare = ['alexnet', 'resnet18', 'vgg16']
learning_rates = [1e-4, 1e-3, 1e-2]
epochs_list = [5, 10, 20]

results_rows = []
run_data = {}

run_idx = 0
total_runs = len(networks_to_compare) * len(learning_rates) * len(epochs_list)

for net_name in networks_to_compare:
    for lr in learning_rates:
        for ep in epochs_list:
            run_idx += 1
            print(f'\n=== Run {run_idx}/{total_runs}: {net_name}, LR={lr}, Epochs={ep} ===')
            try:
                out = train_and_evaluate(net_name, lr, ep)
                if out is None:
                    acc = float('nan')
                else:
                    acc = out['acc']
                    run_data[run_idx] = out
            print(f'Test accuracy: {acc:.4f}' if not np.isnan(acc) else 'Test accuracy: NaN (diverged)')

```



```

except RuntimeError as e:
    print(f'Run FAILED: {e}')
    acc = float('nan')

results_rows.append({
    'Network': net_name, 'LearningRate': lr, 'Epochs': ep,
    'TestAccuracy': acc, 'RunIdx': run_idx,
})

torch.cuda.empty_cache()

results_df = pd.DataFrame(results_rows).sort_values('TestAccuracy', ascending=False, na_position='last')
print('\n', results_df.to_string(index=False))
results_df.to_csv('/kaggle/working/results/accuracy_results.csv', index=False)

```

6. Per-network results

```

best_per_network = {} # net_name -> (best_row, run_result dict)

for net_name in networks_to_compare:
    print('=' * 70)
    print(f'RESULTS FOR: {net_name.upper()}')
    print('=' * 70)

    net_df = results_df[results_df['Network'] == net_name].sort_values(
        'TestAccuracy', ascending=False, na_position='last')
    print(net_df.to_string(index=False))

    net_df_valid = net_df.dropna(subset=['TestAccuracy'])
    if net_df_valid.empty:
        print(f'No successful runs for {net_name} — skipping plots.\n')
        continue

    best_row = net_df_valid.iloc[0]
    best_run_idx = int(best_row['RunIdx'])
    best = run_data[best_run_idx]
    best_per_network[net_name] = (best_row, best)

    classes = best['classes']
    pos_label_idx = classes.index('DR')

    print(f'\nBest config for {net_name}: LR={best_row["LearningRate"]}, "
        f"Epochs={best_row["Epochs"]}, Test Acc={best_row["TestAccuracy"]:.4f}\n")

    fpr, tpr, _ = roc_curve(best['labels'], best['scores'][pos_label_idx], pos_label=pos_label_idx)
    roc_auc = auc(fpr, tpr)

```

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].plot(fpr, tpr, linewidth=2, label=f'AUC = {roc_auc:.3f}')
axes[0].plot([0, 1], [0, 1], linestyle='--', color='gray', linewidth=1)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title(f'ROC Curve — {net_name}\n(LR={best_row["LearningRate"]},
Epochs={best_row["Epochs"]})')
axes[0].legend(loc='lower right')
axes[0].grid(True)

cm = confusion_matrix(best['labels'], best['preds'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)
disp.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title(f'Confusion Matrix — {net_name}')

plt.tight_layout()
plt.savefig(f'/kaggle/working/results/{net_name}_best_roc_confusion.png', dpi=150, bbox_inches='tight')
plt.show()

```

7. Final comparison across all three networks

```

summary = (results_df.groupby('Network')['TestAccuracy']
            .agg(['max', 'mean']).reset_index()
            .rename(columns={'max': 'BestAccuracy', 'mean': 'MeanAccuracy'})
            .sort_values('BestAccuracy', ascending=False))
print(summary.to_string(index=False))
summary.to_csv('/kaggle/working/results/network_comparison_summary.csv', index=False)

plt.figure(figsize=(6, 4.5))
plt.bar(summary['Network'], summary['BestAccuracy'])
plt.ylabel('Best Test Accuracy')
plt.title('Best Test Accuracy by Network Architecture')
plt.ylim(0, 1)
plt.grid(axis='y', alpha=0.3)
plt.savefig('/kaggle/working/results/network_comparison_bar.png', dpi=150, bbox_inches='tight')
plt.show()

# Overlay all three networks' best ROC curves on a single plot
plt.figure(figsize=(6, 6))
for net_name, (best_row, best) in best_per_network.items():
    classes = best['classes']
    pos_label_idx = classes.index('DR')
    fpr, tpr, _ = roc_curve(best['labels'], best['scores'][:, pos_label_idx], pos_label=pos_label_idx)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2, label=f'{net_name} (AUC={roc_auc:.3f})')

```

```

plt.plot([0, 1], [0, 1], linestyle='--', color='gray', linewidth=1)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison — All Networks (Best Config Each)')
plt.legend(loc='lower right')
plt.grid(True)
plt.savefig('/kaggle/working/results/roc_comparison_all_networks.png', dpi=150, bbox_inches='tight')
plt.show()

# Save the single best overall model across all networks
overall_best_net = summary.iloc[0]['Network']
_, overall_best = best_per_network[overall_best_net]
torch.save(overall_best['model'].state_dict(), '/kaggle/working/results/best_model_overall_state_dict.pt')
print(f"\nOverall best network: {overall_best_net} (Best Acc = {summary.iloc[0]['BestAccuracy']:.4f})")
print('All results saved to /kaggle/working/results/')

```